

LangChain + LangGraph DevSecOps Demo

This repository is a **GitHub-ready demo project** that shows a secure CI/CD pipeline integrating **LangChain** and **LangGraph** with DevSecOps controls (linting, security scans, prompt tests, SBOM, container build, and Kubernetes deploy). Use this as a starting point for enterprise pipelines.

Repo layout

```
langchain-langgraph-devsecops-demo/
├── .github/
│   └── workflows/
│       └── ci.yml
├── app/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── README_APP.md
│   ├── main.py
│   ├── chains/
│   │   └── rag_chain.py
│   ├── graphs/
│   │   └── security_graph.py
│   ├── security/
│   │   ├── prompt_security_test.py
│   │   └── bandit.yml
│   └── tests/
│       ├── test_chain.py
│       └── test_graph.py
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── scripts/
│   ├── generate_sbom.sh
│   └── run_local_eval.sh
├── .dockerignore
├── README.md
└── LICENSE
```

Key goals implemented

- Automated unit tests for chains + graphs

- Prompt security test (basic) and template for more advanced checks
 - Static analysis (Bandit)
 - Dependency audit (pip-audit)
 - Build Docker image and generate SBOM via Syft
 - Push image to registry and optionally deploy to Kubernetes
 - Artifacts and logs suitable for LangSmith/observability integration
-

.github/workflows/ci.yml

```
name: CI-CD-LangChain-LangGraph

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    env:
      PYTHONUNBUFFERED: 1

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Cache pip
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements.txt') }}
          restore-keys: |
            ${{ runner.os }}-pip-

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r app/requirements.txt
          pip install bandit pip-audit syft
```

```

- name: Run unit tests
  run: pytest -q app/tests

- name: Run Bandit security scan
  run: bandit -r app/ -c app/security/bandit.yml -f json -o bandit-
report.json || true

- name: Run dependency audit
  run: pip-audit --format json > pip-audit.json || true

- name: Run prompt security tests
  run: python app/security/prompt_security_test.py || true

- name: Run LangChain evaluations (placeholder)
  run: |
    if [ -f app/evals/run_eval.py ]; then
      python app/evals/run_eval.py || true
    else
      echo "No evals present; skipping"
    fi

- name: Build Docker image
  run: |
    docker build -t ${github.repository}:latest app/

- name: Generate SBOM (Syft)
  run: |
    syft ${github.repository}:latest -o json > sbom.json || true

- name: Save artifacts
  uses: actions/upload-artifact@v4
  with:
    name: security-artifacts
    path: |
      bandit-report.json
      pip-audit.json
      sbom.json

docker-push-deploy:
  needs: build-and-test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'

  steps:
    - name: Checkout
      uses: actions/checkout@v4

```

```

- name: Log in to registry
  uses: docker/login-action@v2
  with:
    registry: ${ secrets.REGISTRY_URL }
    username: ${ secrets.REGISTRY_USER }
    password: ${ secrets.REGISTRY_PASSWORD }

- name: Build and push image
  env:
    IMAGE: ${ secrets.REGISTRY_URL }/${ github.repository }:v${
    {{ github.run_number }}
  run: |
    docker build -t $IMAGE app/
    docker push $IMAGE
    echo "IMAGE=$IMAGE" >> $GITHUB_OUTPUT

- name: Deploy to Kubernetes
  if: ${ secrets.KUBE_CONFIG } != ''
  env:
    IMAGE: ${ steps.build-and-push.outputs.IMAGE }
  run: |
    mkdir -p $HOME/.kube
    echo "${ secrets.KUBE_CONFIG }" > $HOME/.kube/config
    kubectl set image deployment/langapp langapp=$IMAGE -n default ||
    kubectl apply -f k8s/deployment.yaml

```

Notes: - Set repository secrets: `REGISTRY_URL`, `REGISTRY_USER`, `REGISTRY_PASSWORD`, and `KUBE_CONFIG` (base64 encoded kubeconfig) in GitHub settings. - The pipeline uploads Bandit/pip-audit/SBOM artifacts so CI can be inspected.

app/requirements.txt

```

fastapi==0.95.2
uvicorn==0.22.0
langchain==0.0.0 # replace with pinned version for your environment
langgraph==0.0.0 # placeholder; use actual package name or local vendoring
pydantic==1.10.9
pytest==7.3.2
python-multipart
sentence-transformers
faiss-cpu
# add other vectorstore clients, langsmith, guardrails, etc.

```

Replace the `langchain` and `langgraph` entries with the correct, pinned package versions you intend to use.

app/Dockerfile

```
FROM python:3.11-slim

ENV PYTHONUNBUFFERED=1
WORKDIR /app

COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

app/main.py (FastAPI minimal example)

```
from fastapi import FastAPI
from chains.rag_chain import create_rag_chain
from graphs.security_graph import SecurityGraph

app = FastAPI()

chain = create_rag_chain()
graph = SecurityGraph()

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/query")
def query(q: str):
    # 1) run prompt sanitization (pre-step)
    # 2) run RAG chain
    # 3) record metadata for audit
    res = chain.run(q)
    # (In production integrate LangSmith tracing here)
    return {"answer": res}
```

app/chains/rag_chain.py (simplified)

```
# Simplified LangChain RAG pipeline skeleton
from typing import Any

class DummyRetriever:
    def get_relevant(self, query: str):
        return ["doc1 content", "doc2 content"]

class DummyLLM:
    def __init__(self):
        pass
    def __call__(self, prompt: str) -> str:
        # replace with a real LLM call wrapper
        return "This is a dummy answer for: " + prompt

def create_rag_chain():
    retriever = DummyRetriever()
    llm = DummyLLM()

    class Chain:
        def run(self, q: str) -> str:
            docs = retriever.get_relevant(q)
            prompt = f"Use the documents:\n{docs}\nAnswer: {q}"
            return llm(prompt)

    return Chain()
```

app/graphs/security_graph.py (LangGraph-style deterministic workflow)

```
# Pseudocode-style LangGraph example. Replace with actual LangGraph API.
from pydantic import BaseModel

class SecurityGraph:
    def __init__(self):
        pass

    def invoke(self, payload: dict):
        # 1. sanitize
        p = self._sanitize(payload)
```

```

    # 2. classify for PII
    piiflag = self._check_pii(p)
    if piiflag:
        return {"error": "PII detected"}
    # 3. call chain
    # 4. validate output schema
    return {"result": "ok"}

def _sanitize(self, payload):
    return payload
def _check_pii(self, payload):
    # implement token-based PII detectors
    return False

```

app/security/prompt_security_test.py

```

# Very small shim to check for dangerous substrings in prompts
DANGEROUS_PATTERNS = ["DROP TABLE", "rm -rf", "sudo", "password=", "cat /etc/passwd"]

TEST_PROMPTS = [
    "Show me how to DROP TABLE users",
    "Please summarize the following document...",
]

import json

failures = []
for p in TEST_PROMPTS:
    for pat in DANGEROUS_PATTERNS:
        if pat.lower() in p.lower():
            failures.append({"prompt": p, "pattern": pat})

if failures:
    print(json.dumps({"failures": failures}))
    raise SystemExit(1)

print("Prompt security checks passed")

```

app/security/bandit.yml

```
exclude:
  - tests
skips:
  - B101 # assert used for tests maybe
```

app/tests/test_chain.py

```
from chains.rag_chain import create_rag_chain

def test_chain_runs():
    chain = create_rag_chain()
    out = chain.run("Hello world")
    assert isinstance(out, str)
    assert "Hello" in out or out != ""
```

app/tests/test_graph.py

```
from graphs.security_graph import SecurityGraph

def test_graph_invoke():
    g = SecurityGraph()
    res = g.invoke({"input": "hello"})
    assert isinstance(res, dict)
```

k8s/deployment.yml (example)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: langapp
spec:
  replicas: 1
  selector:
```



```
matchLabels:
  app: langapp
template:
  metadata:
    labels:
      app: langapp
  spec:
    containers:
      - name: langapp
        image: REPLACE_WITH_IMAGE
        ports:
          - containerPort: 8000
        env:
          - name: LOG_LEVEL
            value: info
```

scripts/generate_sbom.sh

```
#!/usr/bin/env bash
set -euo pipefail
IMAGE="$1"
syft "$IMAGE" -o json > sbom.json
```

README.md (top-level)

```
# LangChain + LangGraph DevSecOps Demo
```

This demo repository demonstrates a secure CI/CD pipeline for LLM-enabled applications built with LangChain and LangGraph.

```
## What it contains
```

- Minimal LangChain RAG chain skeleton
- Minimal LangGraph workflow skeleton
- CI pipeline that runs tests, Bandit, pip-audit, prompt checks
- Dockerfile + SBOM generation
- Kubernetes deployment sample

```
## How to use
```

1. Update ``app/requirements.txt`` with real package versions.
2. Update GitHub secrets for container registry and Kubernetes.
3. Push to ``main`` to run the GitHub Actions workflow.

```
## Security notes
```

- Do NOT commit secrets. Use GitHub secrets for credentials.
- Replace dummy LLMs with actual, secure LLM connectors (with network policies).

Next steps & customization hints

- Replace `DummyLLM` with a secure wrapper (API key from secrets / environment) and ensure network egress restrictions.
- Add LangSmith tracing instrumentation in `main.py` and chain/graph implementations.
- Extend `prompt_security_test.py` to run guardrails/LLM-based prompt injection detectors as part of CI.
- Add more in-depth evals under `app/evals/` using LangChain Evals or custom scripts.
- Add Helm chart if you want production-grade deployments.

Licensing

MIT (example). Adjust as needed.

Generated by ChatGPT — open the files in this canvas to copy or download.